

```

        i++;
    }
}

// Put the pivot on the left side of `items`:
[items[i], items[right]] = [items[right], items[i]];
// END OF THE PARTITIONING

document.write("<pre>....After swapping, the range is: [" + items.slice(left,
right + 1).join(", ") + "]</pre>");
document.write("<pre>Recursively calling quicksort on: [" + items.slice(left,
i).join(", ") + "] and [" + items.slice(i + 1, right + 1).join(", ") + "]</pre
>");

// Call quicksort() on the two partitions:
quicksort(items, left, i - 1); // RECURSIVE CASE
quicksort(items, i + 1, right); // RECURSIVE CASE
}

let myList = [0, 7, 6, 3, 1, 2, 5, 4];
quicksort(myList);
document.write("<pre>[" + myList.join(", ") + "]</pre>");
</script>

```

This code is similar to the code in the binary search algorithm. As defaults, we set the `left` and `right` ends of the range within the `items` array to the beginning and end of the entire array. If the algorithm reaches the base case of the `right` end at or before the `left` end (a range of one or zero items), the sorting is finished ❶.

In each call to `quicksort()`, we partition the items in the current range (defined by the indices in `left` and `right`), and then swap them around so that the items less than the pivot value end up on the left side of the range and the items greater than the pivot value end up on the right side of the range. For example, if 42 is the pivot value in the array [81, 48, 94, 87, 83, 14, 6, 42], a partitioned array would be [14, 6, 42, 81, 48, 94, 87, 83]. Note that a partitioned array is not the same thing as a sorted one: although the two items to the left of 42 are less than 42, and the five items to the right of 42 are greater than 42, the items are not in sorted order.

The bulk of the `quicksort()` function is the partitioning step. To get an idea of how partitioning works, imagine an index `j` that begins at the left end of